



UNIVERSIDAD PERUANA DEL NORTE

EVALUCION T1

SEMANA 5

CURSO:

TECNICA DE PROGRAMACION ORIENTADA A OBJETOS

TEMA:

DESARROLLO DE CASOS

GRUPO 07

INTEGRANTES:

ALVINO ALVAREZ, JORGE LUIS

CODIGO DE ESTUDIANTE:

N00442478

DOCENTE:

MARTIN EDUARDO TORRES RODRIGUEZ

FECHA DE ENTREGA:

DOMINGO 21 DE JUNIO DEL 2026



CONTENIDO

1	EVALUACION T1:	1
2	DESARROLLO DE LOS CASOS	2
2.1	Caso 1: Fundamentos de POO	2
2.1.1	Encapsulamiento:	2
2.1.2	Uso de constructores:	2
2.1.3	Código:.....	3
2.1.4	Construcción de programa:	4
2.2	Caso 2: Métodos, Static y Excepciones	5
2.2.1	Análisis: uso de static y manejo de errores.	5
2.2.2	Código:.....	5
2.3	Caso 3: UML y Análisis	8
2.3.1	Requerimientos Funcionales:	8
2.3.2	Requerimientos no funcionales:.....	9
2.3.3	Historia de usuario:	9
2.3.4	Describir diagrama de clases	10
2.3.5	Análisis: importancia del análisis previo.	14
2.4	Caso 4: Herencia y Polimorfismo	14
2.4.1	Análisis: ventajas de reutilización.	14
2.4.2	Código:.....	14
2.5	Link de Repositorio en GITHUB:.....	18



1 EVALUACION T1:

T1 – Primera Evaluación: Programación Orientada a Objetos

Indicaciones Generales

Trabajo grupal (máx. 5 integrantes) Duración: 1 semana (lunes a domingo 11:59 pm)
Entregables: - Código en Java - Informe en PDF No se permite uso de imágenes Debe incluir análisis por cada caso

Caso 1: Fundamentos de POO

Crear clase Estudiante con atributos privados: nombre, edad, carrera. Implementar constructor, métodos get/set y método para mostrar datos. Registrar 3 estudiantes. Análisis: explicar encapsulamiento y uso de constructores.

Caso 2: Métodos, Static y Excepciones

Crear clase CuentaBancaria con saldo privado. Métodos depositar y retirar con validación. Usar static para contador. Manejar excepciones. Análisis: uso de static y manejo de errores.

Caso 3: UML y Análisis

Definir 5 requerimientos funcionales y 2 no funcionales. Crear 3 historias de usuario con criterios. Describir diagrama de clases. Análisis: importancia del análisis previo.

Caso 4: Herencia y Polimorfismo

Crear clase Persona. Clases hijas Estudiante y Docente. Aplicar herencia y polimorfismo. Análisis: ventajas de reutilización.

Entregables

Word, PDF con:

Código funcional Informe con análisis y conclusiones

2 DESARROLLO DE LOS CASOS

2.1 Caso 1: Fundamentos de POO

Crear clase Estudiante con atributos privados: nombre, edad, carrera. Implementar constructor, métodos get/set y método para mostrar datos. Registrar 3 estudiantes. Análisis: explicar encapsulamiento y uso de constructores.

Análisis: explicar encapsulamiento y uso de constructores:

2.1.1 Encapsulamiento:

El encapsulamiento es un principio fundamental para la programación lineal, esto debido a que agrupa los atributos y métodos de una clase para protegerlos y no dar accesos a ellos desde fuera. Además, esto sirve para brindar seguridad a los datos, dar mejor control y permite que se pueda aplicar validaciones.

Para poder aplicar el encapsulamiento, se debe usar el modificador de acceso llamado “Private” que en español es privado. Además, se debe hacer uso de los métodos públicos “Getter y Setter” ya que ellos permitirán leer o modificar los atributos de forma más controlada. Un punto importante es siempre implementar un constructor para inicializar los atributos de la clase y puedes tener varios constructores con diferente parametrización con el objetivo de brindar flexibilidad en la inicialización.

En nuestro programa de ejemplo, la clase Estudiante encapsula los datos de nombre, edad y carrera. También, proporciona métodos públicos “Getter y Setter” para acceder y modificar los datos de forma controlada. Se observa que tiene un constructor el cual inicializa los atributos.

2.1.2 Uso de constructores:

La función general de un constructor es de inicializa los atributos de una clase. Esto la realiza antes de que el objeto este listo para ser usado. Una característica importante es que tienen el mismo nombre que tiene la clase, tampoco tiene tipo de retorno y puede ser de tipo vacío o parametrizado. El constructor vacío permite la inicialización por defecto mientras que el constructor parametrizado permite inicializar los atributos de la clase con los valores específico durante la creación del objeto.

En nuestro programa presente se observa que tiene un constructor parametrizado ya que tiene la parametrización de los tipos de atributos que ingresaran en la creación de los objetos, por ejemplo, se observa que tiene nombre de tipo “String”, edad de tipo entero (int) y carrera de tipo “String”.

2.1.3 Codigo:

Clase: Estudiante

```
//CLASE ESTUDIANTE
package caso1_fundamento_de_poo;

public class Estudiante
{
    // =====Atributos privados o encapsulados=====JLAA
    private String nombre;
    private int edad;
    private String carrera;

    // =====Constructor para iniciar los atributos=====JLAA
    public Estudiante(String nombre, int edad, String carrera)
    {
        this.nombre = nombre;
        this.edad = edad;
        this.carrera = carrera;
    }

    // =====Metodo get y set=====JLAA

    // ---para nombre---
    public String getNombre()
    {
        return nombre;
    }

    public void setNombre(String nombre)
    {
        this.nombre = nombre;
    }

    // ---para edad---
    public int getEdad()
    {
        return edad;
    }

    public void setEdad(int edad)
    {
        this.edad = edad;
    }

    // ---para carrera---
    public String getCarrera()
    {
        return carrera;
    }

    public void setCarrera(String carrera)
    {
        this.carrera = carrera;
    }

    // =====Metodo que ayuda a mostrar los datos=====
    public void muestraDatos()
    {
        System.out.println("=====");
        System.out.println("Nombre: " + nombre);
        System.out.println("Edad: " + edad);
        System.out.println("Carrera: " + carrera);
    }
}
```

```
}
}
```

Clase Principal: PrincipalEstudiante

```
//CLASE PRINCIPAL
package caso1_fundamento_de_poo;

/**
 *
 */
public class PrincipalEstudiante {

    /**
     * @param args
     */
    public static void main(String[] args) {
        // TODO Auto-generated method stub

        // Ingreso de los registros de 3 estudiantes con uso del constructor ---JLAA

        Estudiante estudiante1 = new Estudiante("Jorge", 27, "Ingenieria Sistema");
        Estudiante estudiante2 = new Estudiante("Luis", 21, "Ingenieria Software");
        Estudiante estudiante3 = new Estudiante("July", 20, "Contabilidad");

        System.out.println("=====");
        System.out.println("==Lista de Estudiantes==");

        // Muestra los datos de los estudiantes ---JLAA
        estudiante1.muestraDatos();
        estudiante2.muestraDatos();
        estudiante3.muestraDatos();

        System.out.println("=====");

    }
}
```

2.1.4 Construcción de programa:

1. En un package de nombre **caso1_fundamento_de_poo**, se creó dos clases, uno es clase Estudiante y el segundo es PrincipalEstudiante.
2. En la clase Estudiante:
 - Se define los atributos que para este caso es nombre de tipo “String”, edad de tipo “int” y carrera de tipo “String”. Todos ellos están encapsulados ya que inician con el modificador “private”.
 - Luego se genera el constructor con los parámetros según los atributos para inicializarlos. Se usa “this” para referencial al objeto actual. Por ejemplo, **this.nombre = nombre**, con esto se está indicando que se asigna al atributo **nombre** del objeto el valor del parámetro **nombre** que se recibió.
 - Después, se aplica los métodos “Getter y Setter”.
 - Finalmente se usa el método para mostrar los datos.
3. En la clase PrincipalEstudiante:
 - Se crea los objetos con los datos.

- Finalmente, se hace un llamado de métodos de cada objeto. Por ejemplo, `estudiante1.muestraDatos();`
- 4. Lo queda es dar clic en Run para ejecutar.
- 5. El programa nos mostrará datos de los estudiantes como nombre, edad y carrera.

2.2 Caso 2: Métodos, Static y Excepciones

Crear clase CuentaBancaria con saldo privado. Métodos depositar y retirar con validación. Usar static para contador. Manejar excepciones.

Análisis: uso de static y manejo de errores.

2.2.1 Análisis: uso de static y manejo de errores.

El uso de static es para diversas situaciones como por ejemplo los métodos estáticos usados para ahorrar memoria y mejorar el rendimiento cuando se utiliza el mismo valor o método. También se usa en bloque estáticos para inicializaciones estáticas de una clase ya que el bloque estático se ejecuta una sola vez cuando la clase se carga en memoria. Otro uso es en las variables estáticas ya que se puede compartir entre todas las instancias de una clase, estas son conocidas como variables de clase.

Sobre el manejo de errores, es usado para evitar que el programa se bloquee, esto debido a que si ocurre una excepción, el programa lo captura mostrando un mensaje de error en lugar de cerrarse. En nuestro programa se aprecia el funcionamiento de ello.

2.2.2 Código:

Clase Principal: PrincipalCuentaBancaria

```
package caso2_metodos_static_y_excepciones;

import java.util.Scanner;
/**
 *
 */
public class PrincipalCuentaBancaria {

    /**
     * @param args
     */
    public static void main(String[] args)
    {
        // TODO Auto-generated method stub
        Scanner scan = new Scanner(System.in);

        // =====Manejo de excepciones=====
        try
        {
            // =====Ingresando cantidad de cuentas===== ---JLAA
            System.out.print("¿Cuántas cuentas desea crear? : ");
            int cantCuentas = scan.nextInt();
        }
    }
}
```

```
// =====Creacion de cuentas===== Segun lo solicitado por el usuario ---JLAA
// Uso del bucle FOR
for (int i = 1; i <= cantCuentas; i++)
{
    System.out.println("\n=====");
    System.out.println("    ***Creando cuenta " + i + " ***");
    System.out.println("=====");

    // -----Ingreso de saldo inicial-----
    System.out.print("Ingresar saldo inicial: ");
    double saldoInicial = scan.nextDouble();
    CuentaBancaria cuentaBanca = new CuentaBancaria(saldoInicial);

    // -----Ingreso de deposito-----
    System.out.print("Ingresar monto a depositar: ");
    double deposito = scan.nextDouble();
    cuentaBanca.depositar(deposito);

    // -----Impresion de saldo despues del deposito-----
    System.out.println("-----");
    System.out.println("Saldo actual de la cuenta " + i + ": " +
cuentaBanca.getSaldoCuentaINI()); // Saldo total despues del deposito*
    System.out.println("-----");

    // -----Ingreso monto a retirar-----
    System.out.print("Ingresar monto para retirar: ");
    double retiro = scan.nextDouble();

    // -----Manejo de excepciones para retiro-----
    try
    {
        cuentaBanca.retirar(retiro);
    } catch (Exception e)
    {
        System.out.println("Surgio un error al retirar: " + e.getMessage());
    }

    // -----Impresion saldo final-----
    System.out.println("#####");

    System.out.println("Saldo final de la cuenta " + i + ": " +
cuentaBanca.getSaldoCuenta());

    System.out.println("#####");
    System.out.println(" ");
}

// -----Impresion de cuentas generadas----- Esto se presente despues del las
operaciones de las cuentas
System.out.println("=====");
System.out.println("Cantidad de cuentas generadas: " +
CuentaBancaria.getContadorCuenta());
System.out.println("=====");

} catch (Exception e)
{
    // -----Impresion mensaje de ERROR----- Captura de mensaje de error
    System.out.println("*** ERROR: " + e.getMessage() + " ***");
} finally
{
    scan.close(); // Cierra el Scanner para liberar memoria o recursos
}

}

}
```


Clase: CuentaBancaria

```
package caso2_metodos_static_y_excepciones;

public class CuentaBancaria
{
    //====Atributo privado==== Cada cuenta contiene su saldo ---JLAA
    private double saldoCuenta;
    private double saldoTotDep; //****

    //====Atributo estatico==== Estos es para el contador ---JLAA
    private static int contadorCuenta = 0;

    //====Constructor==== Para inicializar la cuenta con saldo inicial ---JLAA
    public CuentaBancaria(double saldoInicial)
    {
        if (saldoInicial < 0)
        {
            // Condicional para activar la validacion si el saldo inicial en CERO o negativo
            throw new IllegalArgumentException(" Saldo inicial no puede ser negativo");
        }
        this.saldoCuenta = saldoInicial;
        contadorCuenta++;
    }

    //====Metodo depositar para ingresar dinero====
    public void depositar(double montoCuenta)
    {
        // Condicional para activar la validacion si el monto en CERO o negativo
        if (montoCuenta <= 0)
        {
            // Validacion si el monto es menor que cero
            throw new IllegalArgumentException("Su deposito tiene que ser mayor que cero.");
        }
        saldoCuenta = saldoCuenta + montoCuenta; // Operacion suma para el saldo actual
        saldoTotDep = saldoCuenta; // Saldo total despues del deposito*
    }

    //====Metodo depositar para retirar dinero===== ---JLAA
    public void retirar(double montoCuenta) throws Exception
    {
        if (montoCuenta <= 0)
        {
            // Validacion cuando se desea retirar en cero o negativo ---JLAA
            throw new IllegalArgumentException("Su retiro tiene que ser mayor que cero.");
        }
        if (montoCuenta > saldoCuenta)
        {
            throw new Exception("Su Fondo es insuficiente");
        }
        saldoCuenta = saldoCuenta - montoCuenta; // Operacion resta para el saldo actual
    }

    //=====Metodo get===== Usado para consultar saldo
    public double getSaldoCuenta()
    {
        return saldoCuenta;
    }

    //=====Metodo static===== Usado para consultar la cantidad de cuentas
}
```

```

public static int getContadorCuenta()
{
    return contadorCuenta;
}

// =====Metodo get===== Usado para consultar saldo inicial despues de depositar
public double getSaldoCuentaINI() //Saldo total despues del deposito*
{
    return saldoTotDep; // Saldo total despues del deposito*
}
}

```

2.3 Caso 3: UML y Análisis

Definir 5 requerimientos funcionales y 2 no funcionales. Crear 3 historias de usuario con criterios. Describir diagrama de clases. Análisis: importancia del análisis previo.

2.3.1 Requerimientos Funcionales:

ID	Categoría	Descripción
RF-01	Manejo de cliente	El sistema permitirá hacer registros clientes nuevos en la base de datos y almacena por nombre, apellido, DNI, dirección, numero de contacto, etc. No permitirá registros duplicados cuando el cliente ya exista
RF-02	Manejo de Inventarios	El sistema ayudará en el control de inventario como la creación de nuevos productos, cambiar el estado al dar de baja a códigos no vigentes, los ingresos y salidas de productos por la compra realizada o por merma.
RF-03	Proceso de venta	El sistema permitirá realizar registros de ventas el cual estará asociado a un cliente activo en el sistema, según la fecha actual y producto vigente imprimiéndolo en un archivo PDF y XML para ser enviado al cliente.
RF-04	Reportes	El sistema permitirá generar reportes como registro de ventas, historial, movimientos contables según el filtro que se use como fecha, tipo de registro o tipo de documento.
RF-05	Gestión de proveedores	El sistema permitirá realizar registros de nuevos proveedores en la base de datos, almacenando por nombre, apellido, DNI, dirección, numero de contacto, etc. No permitirá registros duplicados cuando el proveedor ya exista.

2.3.2 Requerimientos no funcionales:

ID	Categoría	Descripción
RNF-01	Portabilidad	El sistema desarrollado en Java con Eclipse debe poder ejecutarse en cualquier sistema operativo sea Linux, MacOS o Window.
RNF-02	Usabilidad	La interfaz grafica desarrollada mediante WindowBuilder debe mostrar los datos de las personas en un formato alineado, legible y con una vista amigable e intuitivo.
RNF-03	Seguridad de datos	El sistema debe restringir el acceso a realizar modificaciones de registros de ventas como los importes, sea en moneda local o en divisa en dólares, eliminación de productos, etc. Solo el usuario administrativo autorizado puede tener acceso a realizar alguna modificación.

2.3.3 Historia de usuario:

Historia de usuario ID	Título	Descripción	Criterios de aceptación
HU1	Generación de reporte	Como usuario administrativo, Quiero poder obtener un reporte de los registros de ventas actuales del mes de junio del 2026, Para realizar un análisis mensual.	Escenario 1: Reporte generado éxitos. Según los filtros seleccionados y completados puedo obtener un reporte de ventas exacto para mi análisis. Escenario 2: Reporte fallido. Cuando no uso los filtros me propone un reporte desde inicio de las ventas de meses anteriores y antes de generar me indica no uso los filtros. ¿Desea proseguir o no? Si le doy que no, no genera el reporte y puedo usar los filtros de nuevo.
HU2	Registro de cliente	Como usuario administrativo, Quiero poder ingresar los datos personales de los clientes nuevos Para que quede registrado en el sistema.	Escenario 1: Registro exitoso. Esto sucede cuando se registra un nuevo cliente completando los campos importantes y principales los cuales son necesarios para el registro. Escenario 2: Registro fallido. Esto sucede cuando no se ha completado los campos necesarios para realizar el registro o ya exista en el sistema.
HU3	Gestión de inventario	Como: Usuario de almacén, Quiero realizar el ingreso de mercadería entregada por el proveedor	Escenario 1: Registro del ingreso de mercadería exitoso. Cuando ingreso el documento de remisión y registro en el sistema completando todos los

		Para que en el sistema tenga Stock de venta y pueda realizar ventas.	campos, el sistema me permite realizar el registro. Escenario 2: Registro fallido. Esto ocurre cuando no se completa los campos importantes como por ejemplo el nombre del proveedor o la línea de producto este vacío, etc.
--	--	--	---

2.3.4 Describir diagrama de clases

1: Módulo de actores que participan en la herencia:

Se genera las siguientes clases:

- Clase persona: Esta será la clase Padre. Esta actuara como una super clase que encapsulara los atributos comunes como nombre, apellido, DNI, dirección, numero de contacto, etc.
- Clase Cliente: Esta clase heredará de persona sus atributos necesarios para el registro y contiene sus propios métodos. Esta contendrá la lógica de validación en su constructor o método como “registrar()” que servirá para verificar que el DNI no exista previamente y así evitar duplicados.
- Clase Proveedor: Esta hereda de Persona sus atributos y almacena los datos del proveedor bajo la misma lógica de restricción de duplicados como lo tiene para cliente.

Todos estos cubren los requerimientos funcionales RF-01 y RF-05.

2: Modulo de Inventario:

Se genera la clase Producto:

- La clase Producto: Esta servirá para gestionar el inventario. Sus atributos incluyen código, descripción, stock y estado de Activo o Baja. Sus métodos permitirán dar de alta o de baja cambiando el estado y actualizar el stock como ingresoMercaderia() y salida PorVenta().

Esta cubre el requerimiento RF-02

3: Modulo de ventas:

Se genera las clases:

- Clase DetalleVenta: Representa la línea de un producto en la venta. Relaciona un Producto con una cantidad y un precio, sea de moneda local o divisa en dólares.
- Clase Venta: Esta es la clase central del proceso. Esta tiene una composición con DetalleVenta, debido a que una venta está compuesta por varios detalles, y a una asociación con Cliente y Fecha.

- El módulo cuenta con métodos específicos como registrarVenta() y generarComprobante(), el cual invoca a clases exportadoras para crear los archivos físicos de formato PDF o XML según lo requerido.

Con esto se cubre el requerimiento RF-03.

4: Modulo de Reporte:

Para esto se hará uso de una clase llamada GestorReportes que actuará como un servicio que recibe parámetros del filtro como puede ser fecha, tipo de documento, y consulta la lista de objetos Venta para poder generar los historiales y movimientos contables que se hayan solicitado.

Esto cubre al requerimiento RF-04.

5: Modulo de Seguridad y control:

Para este módulo se hará uso de la clase Usuario y Enum Rol. Para poder restringir las modificaciones de importes o eliminación de productos, se hará uso de la clase Usuario ya que tiene un atributo de tipo Rol el cual puede ser Administrativo, Almacen, etc. Además, los métodos críticos de las clases Venta y Producto requerirán en sus parámetros a un objeto Usuario con el objetivo de validar sus permisos antes de ejecutar la acción.

Con esto se cubre el requerimiento RNF-03.

6: La relación que tienen:

- **Relaciones de Herencia:**

Persona <I—Cliente (RF-01): La clase Cliente hereda los atributos comunes de la super clase Persona como nombre, apellido, DNI, etc.

Persona <I—Proveedor (RF-05): Proveedor hereda de Persona reutilizando el código y evitando la duplicación de estructura de información o datos.

- **Relación de composición:**

Venta <*>—DetalleVenta (RF-03): Esta es una relación fuerte de todo-parte. Esto debido a que una Venta esta compuesta por uno o varios DetalleVenta (1..*). Una venta no puede existir sin sus detalles. Si se elimina la venta, los detalles de esa venta también se eliminan automáticamente de la memoria del sistema.

- **Relación de Asociación:**

Venta ---Cliente (RF-03): Una Venta (1..*) pertenece a un solo Cliente (1). El cliente de estar activo en el sistema según lo requerido en RF03 para que la asociación se pueda crear.

Proveedor---Producto (RF-02): Un Proveedor puede suministrar o estar asociado a múltiples Productos. Esta relación es consultada cuando el usuario de almacén registra el ingreso de la mercadería al almacén.

Usuario---Venta y Producto (RNF-03): Un usuario que tenga rol Administrativo es el responsable de modificar múltiples registros de Venta o dar de baja a Productos. La asociación garantiza que el sistema verifique los permisos antes de ejecutar la acción.

- Relación de dependencia

GestorReportes ----> Venta (RF-04): La clase GestorReportes depende de Venta. No es propiedad de la venta, sino que temporalmente toma una lista de objetos Venta para aplicarle los filtros que puede ser por fecha, tipo de documento y luego ellos empieza a generar el análisis mensual.

Venta ---->Usuario (RNF-03): Esto es cuando se intenta modificar un importe de dólares o moneda local de una venta ya registrada, el método de Venta depende de recibir un objeto Usuario temporalmente para poder verificar si tiene el rol de Administrativo antes de autorizar el cambio.

Venta ----> GeneradorArchivos (RF-03): La clase Venta depende de un servicio o clase auxiliar para convertir el texto en los archivos físicos en PDF y XML.

Descripción gráfica:

Usuario
- usuario: string
- clave: string
- rol: Rol
+ tienePermisos(tipoAccion: string)

Rol
ADMINISTRATIVO
ALMACEN

GestorReportes
+ generarReporteVentas(filtros: Map): Lista<Venta>

+ generarHistorial(cliente: Cliente): Lista<Venta>
--

Venta
- fecha: Date
- estado: String
+ registrarVenta(cliente: Cliente): boolean
+ generarComprobantePDF(): File
+ generarComprobanteXML(): File

DetalleVenta
- cantidad: int
- precioLocal: double
- precioDolar: double
+ calcularSubtotal(): double

Persona
- nombre: String
- apellido: String
- DNI: String
- dirección: String
- numeroContacto: String
+ getDatosCompleto(): String

Cliente
+ registrar(): boolean

Proveedor
+ registrar(): boolean

Producto
- código : String
- descripcion: String
- stock: int
- estado: String
+ darDeBaja(): void
+ actualizarStock(Cantidad: int, tipoMov: String): void

2.3.5 Análisis: importancia del análisis previo.

Para una implementación de software orientado a objetos es importante el análisis previo porque permite modelar una solución al problema presentado y mediante los requerimientos y el uso de UML antes de escribir una sola línea de código en Java o cualquier otro lenguaje. La importancia del análisis radica por lo siguiente:

- Prevención de errores costosos.
- Definición de la arquitectura de seguridad.
- Manejo de complejidad mediante composición.
- Escalabilidad frente a nuevos requerimientos.

2.4 Caso 4: Herencia y Polimorfismo

Crear clase Persona. Clases hijas Estudiante y Docente. Aplicar herencia y polimorfismo. Análisis: ventajas de reutilización.

2.4.1 Análisis: ventajas de reutilización.

Una de las ventajas que se tiene al usar la herencia el cual es la reutilización es tener menos duplicación de código, esto debido a que la lógica se define una sola vez. Además, el uso se puede extender, por ejemplo, si se crea una nueva clase hija, se reutiliza los atributos y se sobre escribe como se puede ver en el ejemplo de nuestro código. Un punto importante es que el mantenimiento es fácil ya que, si se modifica la clase base, en las demás hijas se refleja el cambio.

2.4.2 Código:

Clase Principal: PrincipalHerenciaPolimorfismo

```
package caso4_herencia_y_poliformismo;

import padre_persona.Persona; // Se usa para importa para traer la clase Persona del paquete padre_persona
import hijas_estudiante_docente.Estudiante; // Se usa para importa para traer la clase Estudiante del paquete
hijas_estudiante_docente
import hijas_estudiante_docente.Docente; // Se usa para importa para traer la clase Estudiante del paquete
hijas_estudiante_docente

/**
 *
 */
// Clase Principal - Aqui se muestra la Herencia y Polimorfismo
public class PrincipalHerenciaPoliformismo
{

    /**
     * @param args
     */
}
```



```

public static void main(String[] args)
{
    // TODO Auto-generated method stub

    //Poliformismo de asignacion
    Persona PEstud = new Estudiante();
    Persona PDocent = new Docente();

    // Intanciacion o creacion de objetos
    Persona Person = new Persona ("Jor", "Alv", 21, 777777777);
    Estudiante Estud = new Estudiante ("Luis", "Alva", 18, 33333333, "Ingenieria", "5");
    Docente Docent = new Docente ("Raul", "Alvarez", 35, 44444444, "Matematicas", "UPN" );

    // Invocacion de metodo PERSONA (De la Clase Padre)
    System.out.println("=====");
    System.out.println("=====PERSONA=====");
    System.out.println("=====");

    System.out.println("-----Datos-----");
    System.out.println(Person.mostrarDatosPersona());
    System.out.println("");
    System.out.println("-----Mensaje-----");
    System.out.println(Person.actividad());
    System.out.println("");

    // Invocacion de metodo del objeto ESTUDIANTE (De la clase Hija)
    System.out.println("=====");
    System.out.println("=====ESTUDIANTE=====");
    System.out.println("=====");
    // Herencia : Datos heredados mas los propios
    System.out.println("----Datos-Heredados-----");
    System.out.println(Estud.mostrarDatosCompleto());
    System.out.println("");
    // Polimorfismo en Estudiante
    System.out.println("-----Polimorfismo-----");
    System.out.println(PEstud.actividad());
    System.out.println("");

    // Invocacion de metodo del objeto DOCENTE (De la clase Hija)
    System.out.println("=====");
    System.out.println("=====DOCENTE=====");
    System.out.println("=====");
    // Herencia : Datos heredados mas los propios
    System.out.println("----Datos-Heredados-----");
    System.out.println(Docent.mostrarDatosCompleto());
    System.out.println("");
    // Polimorfismo en Estudiante
    System.out.println("-----Polimorfismo-----");
    System.out.println(PDocent.actividad());
    System.out.println("");

}
}

```

Clase Padre: Clase Persona

```
package padre_persona;
```

```

/**
 *
 */

```

```
// >>> Clase Persona es la clase Padre <<<
public class Persona
{
    // >>> Encapsulamiento <<< JLAA
    // >>> Atributos protegidos <<< JLAA
    protected String nombre, apellido;
    protected int edad, DNI;

    // >>> Constructor vacio <<< JLAA
    public Persona()
    {

    }

    // >>> Constructor parametrizado <<< JLAA
    public Persona(String nombre, String apellido, int edad, int DNI )
    {
        this.nombre = nombre;
        this.apellido = apellido;
        this.edad = edad;
        this.DNI = DNI;
    }

    // >>> Metodo que muestra los datos de la clase persona <<< JLAA
    public String mostrarDatosPersona()
    {
        return
            "Nombre : " + nombre + "\n" +
            "Apellido: " + apellido + "\n" +
            "Edad    : " + edad + "\n" +
            "DNI     : " + DNI;
    }

    // >>> Metodo que muestra mensaje <<< JLAA
    public String actividad()
    {
        return ">>> Realizo una actividad <<<";
    }
}
```

Clase Hija: Estudiante

```
package hijas_estudiante_docente;

import padre_persona.Persona; // Se usa para importa para traer la clase Persona del paquete padre_persona
/**
 *
 */

// >>> Clase Estudiante es la clase hija de la clase Padre Persona <<<
public class Estudiante extends Persona
{
    // >>> Atributos de la clase Estudiante <<< JLAA
    private String carrera;
    private String ciclos;

    // >>> Constructor vacio <<< JLAA
    public Estudiante()
    {

    }
}
```

```
// >>> Constructor parametrizado <<< JLAA
public Estudiante(String nombre, String apellido, int edad, int DNI, String carrera, String ciclos)
{
    super(nombre, apellido, edad, DNI);
    this.carrera = carrera;
    this.ciclos = ciclos;
}

// >>> Metodo que muestra los datos de la clase Estudiante <<< JLAA
public String mostrarDatosCompleto()
{
    return
        mostrarDatosPersona() + "\n----Datos-Propios-----" + "\n" +
        "Carrera del Estudiante      : " + carrera + "\n" +
        "Ciclo de Estudios : " + ciclos;
}

// >>> Metodo que muestra mensaje <<< JLAA
public String actividad()
{
    return ">>> Realizo una actividad el cual es Estudiar en la universidad <<<";
}
}
```

Clase Hija: Docente

```
package hijas_estudiante_docente;

import padre_persona.Persona; // Se usa para importa para traer la clase Persona del paquete padre_persona
/**
 *
 */

// >>> Clase Docentete es la clase hija de la clase Padre Persona <<<
public class Docente extends Persona
{
    // >>> Atributos de la clase Docente <<< JLAA
    private String asignatura;
    private String universidad;

    // >>> Constructor vacio <<< JLAA
    public Docente()
    {
    }

    // >>> Constructor parametrizado <<< JLAA
    public Docente(String nombre, String apellido, int edad, int DNI, String asignatura, String universidad)
    {
        super(nombre, apellido, edad, DNI);
        this.asignatura = asignatura;
        this.universidad = universidad;
    }

    // >>> Metodo que muestra los datos de la clase Docente <<< JLAA
    public String mostrarDatosCompleto()
    {
        return
    }
}
```

```
+
                                mostrarDatosPersona() + "\n" + "\n-----Datos-Propios-----" + "\n"
+
                                "Asignatura del docente      : " + asignatura + "\n" +
                                "Universidad del docente   : " + universidad;
                                }

// >>> Metodo  que muestra mensaje <<<  JLAA
public String actividad()
{
    return ">>> Realizo una actividad el cual es trabajar como docente en la universidad <<<";
}

}
```

2.5 Link de Repositorio en GITHUB:

https://github.com/JorgeLAA777/Practicas-de-campo-C5-TPOO-grupo07-Individual/tree/main/PC05_Semana_5/T1/Evaluacion_T1/src